

Cost Manager - Final Project Submission

Team Information

Team Manager: [Enter First Name] [Enter Last Name]

Team Members:

1. [First Name] [Last Name] - ID: [ID] - Mobile: [Number] - Email: [Email]
2. [First Name] [Last Name] - ID: [ID] - Mobile: [Number] - Email: [Email]

Project Links

Video Demonstration: [Insert YouTube Link Here]

Live Deployment: <https://cost-manager-0w4h.onrender.com/>

Collaborative Tools Summary

We utilized GitHub for version control and collaborative code management, allowing team members to work on separate features concurrently while maintaining a stable main branch. Slack was used as our primary communication channel for daily syncs, sharing code snippets, and discussing UI architecture decisions.

Source Code

db.js (Vanilla Version)

```
/*
 * db.js - Cost Manager LocalStorage Library (Vanilla JavaScript Version)
 * This library wraps localStorage to manage cost items and generate financial reports.
 */

// Initialize the global db namespace
window.db = {};

/*
 * Database instance object prototype
 */
window.db.Database = function(databaseName) {
    // Store the database name for localStorage key management
    this.databaseName = databaseName;
    this.storageKey = 'db_' + databaseName;
};

/*
 * Initialize or open an existing database
 * @param {string} databaseName - Name of the database
 * @param {number} databaseVersion - Version number of the database
 * @return {object} Database instance with addCost and getReport methods
 */
window.db.openCostsDB = function(databaseName, databaseVersion) {
    // Create database instance
    const dbInstance = new window.db.Database(databaseName);

    // Initialize storage if it doesn't exist
    if (!localStorage.getItem(dbInstance.storageKey)) {
        localStorage.setItem(dbInstance.storageKey, JSON.stringify({
            version: databaseVersion,
            costs: []
        }));
    }
}

/*
 * Add a new cost item to the database
 * @param {object} cost - Object with sum, currency, category, description
 * @return {object} The newly added cost item with date
 */
dbInstance.addCost = function(cost) {
    // Retrieve current database state
    const dbData = JSON.parse(localStorage.getItem(this.storageKey));

    // Create new cost object with timestamp
    const newCost = {
        id: Date.now() + Math.random(),
        sum: cost.sum,
        currency: cost.currency,
        category: cost.category,
        description: cost.description,
        date: new Date().toISOString()
    };

    // Add cost to costs array
    dbData.costs.push(newCost);
};
```

```

        // Persist updated database to localStorage
        localStorage.setItem(this.storageKey, JSON.stringify(dbData));

        // Return cost object (without internal id)
        return {
            sum: newCost.sum,
            currency: newCost.currency,
            category: newCost.category,
            description: newCost.description
        };
    };

    /**
     * Generate a report for a specific month and year in a selected currency
     * @param {string} currency - Currency code (USD, GBP, EURO, ILS)
     * @param {number} year - Year (optional, defaults to current year)
     * @param {number} month - Month 1-12 (optional, defaults to current month)
     * @return {object} Report object with costs array and total
     */
    dbInstance.getReport = function(currency, year, month) {
        // Use current date if year and month not provided
        const now = new Date();
        const reportYear = year || now.getFullYear();
        const reportMonth = month || (now.getMonth() + 1);

        // Retrieve all costs from storage
        const dbData = JSON.parse(localStorage.getItem(this.storageKey));
        const allCosts = dbData.costs || [];

        // Filter costs by target month and year
        const filteredCosts = allCosts.filter(function(cost) {
            const costDate = new Date(cost.date);
            return costDate.getFullYear() === reportYear &&
                (costDate.getMonth() + 1) === reportMonth;
        });

        // Format costs for report output (include original currency, day only)
        const reportCosts = filteredCosts.map(function(cost) {
            const costDate = new Date(cost.date);
            return {
                sum: cost.sum,
                currency: cost.currency,
                category: cost.category,
                description: cost.description,
                date: { day: costDate.getDate() }
            };
        });

        // Calculate total sum in original currencies
        let totalSum = 0;
        for (let i = 0; i < reportCosts.length; i++) {
            totalSum += reportCosts[i].sum;
        }

        // Return report object with target currency and totals
        return {
            year: reportYear,
            month: reportMonth,
            costs: reportCosts,
            total: {
                currency: currency,
                sum: totalSum
            }
        };
    };

    return dbInstance;
};

/**
 * Global reference to the last opened database instance
 * This allows getReport to be called on window.db directly
 */
window.db._currentInstance = null;

/**
 * Get report from the current/last opened database
 * This static method allows calling db.getReport() directly
 * @param {string} currency - Currency code (USD, GBP, EURO, ILS)
 * @param {number} year - Year (optional, defaults to current year)
 * @param {number} month - Month 1-12 (optional, defaults to current month)
 * @return {object} Report object with costs array and total
 */
window.db.getReport = function(currency, year, month) {
    // Use the last opened database instance

```

```

    if (window.db._currentInstance && window.db._currentInstance.getReport) {
        return window.db._currentInstance.getReport(currency, year, month);
    }

    // Return empty report if no database opened
    const now = new Date();
    return {
        year: year || now.getFullYear(),
        month: month || (now.getMonth() + 1),
        costs: [],
        total: { currency: currency, sum: 0 }
    };
};

// Update openCostsDB to store reference to current instance
window.db._originalOpenCostsDB = window.db.openCostsDB;
window.db.openCostsDB = function(databaseName, databaseVersion) {
    const instance = window.db._originalOpenCostsDB(databaseName, databaseVersion);
    window.db._currentInstance = instance;
    return instance;
};
``n
## App.js
`javascript
import React, { useState } from 'react';
import { Container, CssBaseline, AppBar, Toolbar, Typography, Tabs, Tab, Box } from '@mui/material';
import AddCost from './pages/add_cost';
import Reports from './pages/reports';
import Settings from './pages/settings';

// TabPanel component for hiding/showing tab content
function TabPanel(props) {
    const { children, value, index, ...other } = props;
    return (
        <div
            role="tabpanel"
            hidden={value !== index}
            id={`tabpanel-${index}`}
            aria-labelledby={`tab-${index}`}
            {...other}
        >
            {value === index && <Box>{children}</Box>}
        </div>
    );
}

// Main App component with tab-based navigation
function App() {
    // State to track active tab
    const [tabValue, setTabValue] = useState(0);

    // Handle tab selection changes
    const handleTabChange = (event, newValue) => {
        setTabValue(newValue);
    };

    // Render app with header and tabbed interface
    return (
        <React.Fragment>
            {/* Application header with title */}
            <CssBaseline />
            <AppBar position="static" sx={{ mb: 4 }}>
                <Toolbar>
                    <Typography variant="h6">Cost Manager</Typography>
                </Toolbar>
            </AppBar>
            {/* Main container with tab-based navigation */}
            <Container>
                <Box sx={{ borderBottom: 1, borderColor: 'divider' }}>
                    <Tabs value={tabValue} onChange={handleTabChange} aria-label="navigation tabs">
                        <Tab label="Add Cost" id="tab-0" aria-controls="tabpanel-0" />
                        <Tab label="Reports" id="tab-1" aria-controls="tabpanel-1" />
                        <Tab label="Settings" id="tab-2" aria-controls="tabpanel-2" />
                    </Tabs>
                </Box>
                {/* Tab panels for different sections of the application */}
                <TabPanel value={tabValue} index={0}>
                    <AddCost />
                </TabPanel>
                <TabPanel value={tabValue} index={1}>
                    <Reports />
                </TabPanel>
                <TabPanel value={tabValue} index={2}>
                    <Settings />
                </TabPanel>
            </Container>
        </React.Fragment>
    );
}

```

```

    </React.Fragment>
  );
}

export default App;
`n
## index.js
`javascript
// Entry point for the React application - renders the main App component
import React from 'react';
import ReactDOM from 'react-dom/client';
import { CssBaseline } from '@mui/material';
import App from './app';

// Create root element and render the application
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    { /* CssBaseline helps normalize styles as per professional standards */ }
    <CssBaseline />
    <App />
  </React.StrictMode>
);
`n

## add_cost.js
`javascript

import React, { useState } from 'react';
import { TextField, Button, MenuItem, Box, Typography, Paper, Alert } from '@mui/material';
import db from '../services/db';

// Predefined categories for cost items
const categories = ['Food', 'Health', 'Housing', 'Transport', 'Education', 'Other'];
// Supported currencies for conversion
const currencies = ['USD', 'ILS', 'GBP', 'EURO'];

const AddCost = () => {
  // Form state management with default values
  const [form, setForm] = useState({ sum: '', currency: 'USD', category: 'Food', description: '' });
  // State for displaying success/error messages
  const [status, setStatus] = useState(null);

  // Handle form field changes
  const handleChange = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };

  // Handle form submission - add cost to database
  const handleSubmit = (e) => {
    e.preventDefault();
    try {
      // Open database (synchronous)
      const ob = db.openCostsDB("costsdb", 1);
      // Add the cost (Convert sum to a Number)
      ob.addCost({ ...form, sum: Number(form.sum) });

      // Show success message and reset form
      setStatus({ type: 'success', msg: 'Cost item added successfully!' });
      setForm({ sum: '', currency: 'USD', category: 'Food', description: '' });
    } catch (err) {
      // Show error message if submission fails
      setStatus({ type: 'error', msg: 'Failed to add cost: ' + err });
    }
  };

  // Render form UI with MUI components
  return (
    <Paper elevation={3} sx={{ p: 4, maxWidth: 500, mx: 'auto', mt: 5 }}>
      <Typography variant="h5" gutterBottom>Add New Cost Item</Typography>
      {status && <Alert severity={status.type} sx={{ mb: 2 }}>{status.msg}</Alert>}

      <Box component="form" onSubmit={handleSubmit} sx={{ display: 'flex', flexDirection: 'column', gap: 2 }}>
        <TextField label="Sum" name="sum" type="number" value={form.sum} onChange={handleChange} required fullWidth />

        <TextField select label="Currency" name="currency" value={form.currency} onChange={handleChange} fullWidth>
          {currencies.map((c) => <MenuItem key={c} value={c}>{c}</MenuItem>)}
        </TextField>

        <TextField select label="Category" name="category" value={form.category} onChange={handleChange} fullWidth>
          {categories.map((cat) => <MenuItem key={cat} value={cat}>{cat}</MenuItem>)}
        </TextField>

        <TextField label="Description" name="description" value={form.description} onChange={handleChange} required fullWidth
      </Box>
    </Paper>
  );
};

```

```

        <Button type="submit" variant="contained" size="large" sx={{ mt: 2 }}>
          Add Item
        </Button>
      </Box>
    </Paper>
  );
};

export default AddCost;

``n
## reports.js
`javascript

import React, { useState, useEffect } from 'react';
import { Paper, Box, TextField, MenuItem, Button, Typography, Grid } from '@mui/material';
import db from '../services/db';
import exchangeRateService from '../services/exchange_rate_service';
import PieChart from '../components/Charts/pie_chart';
import BarChart from '../components/Charts/bar_chart';

// Supported currencies for reporting
const currencies = ['USD', 'ILS', 'GBP', 'EURO'];

const Reports = () => {
  // Initialize with current date values
  const currentDate = new Date();
  // State management for filters and display data
  const [month, setMonth] = useState(currentDate.getMonth() + 1);
  const [year, setYear] = useState(currentDate.getFullYear());
  const [currency, setCurrency] = useState('USD');
  // State for report data and UI feedback
  const [reportData, setReportData] = useState(null);
  const [yearlyData, setYearlyData] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);
  // State for exchange rates used in conversions
  const [exchangeRates, setExchangeRates] = useState(exchangeRateService.DEFAULT_RATES);

  // Load exchange rates on component mount
  useEffect(() => {
    const loadRates = async () => {
      const rates = await exchangeRateService.fetchExchangeRates();
      setExchangeRates(rates);
    };
    loadRates();
  }, []);

  // Helper function to convert costs to selected currency
  const convertCost = (amount, fromCurrency, toCurrency) => {
    return exchangeRateService.convertCurrency(amount, fromCurrency, toCurrency, exchangeRates);
  };

  // Fetch monthly report data and convert costs to selected currency
  const generateReport = () => {
    setLoading(true);
    setError(null);
    try {
      // Open database and fetch report for selected month/year (synchronous)
      const ob = db.openCostsDB('costsdb', 1);
      const report = ob.getReport(currency, year, month);

      // Convert all costs to selected currency and add conversion metadata
      const convertedCosts = report.costs.map(cost => ({
        ...cost,
        convertedSum: convertCost(cost.sum, cost.currency, currency),
        originalCurrency: cost.currency,
        originalSum: cost.sum
      }));

      // Calculate total in selected currency
      const totalInSelectedCurrency = convertedCosts.reduce((sum, cost) => sum + cost.convertedSum, 0);

      // Update report state with converted data
      setReportData({
        ...report,
        costs: convertedCosts,
        total: { currency, sum: totalInSelectedCurrency }
      });
    } catch (err) {
      setError('Failed to generate report: ' + err);
      console.error(err);
    } finally {
      setLoading(false);
    }
  };
};

```

```

// Fetch yearly data for bar chart (all 12 months)
const generateYearlyReport = () => {
  setLoading(true);
  setError(null);
  try {
    // Open database connection (synchronous)
    const ob = db.openCostsDB('costsdb', 1);
    // Object to store monthly totals
    const monthlyTotals = {};

    // Fetch data for all 12 months and convert to selected currency
    for (let m = 1; m <= 12; m++) {
      const report = ob.getReport(currency, year, m);
      // Sum costs for this month, converting to selected currency
      const totalInSelectedCurrency = report.costs.reduce((sum, cost) => {
        return sum + convertCost(cost.sum, cost.currency, currency);
      }, 0);
      monthlyTotals[m] = totalInSelectedCurrency;
    }

    setYearlyData(monthlyTotals);
  } catch (err) {
    setError('Failed to generate yearly report: ' + err);
    console.error(err);
  } finally {
    setLoading(false);
  }
};

// Auto-generate reports when any filter parameter changes
useEffect(() => {
  generateReport();
  generateYearlyReport();
}, [month, year, currency]);

// Month names for display
const monthNames = ['', 'January', 'February', 'March', 'April', 'May', 'June',
  'July', 'August', 'September', 'October', 'November', 'December'];

// Year options for dropdown (current year +/- 2 years)
const years = Array.from({ length: 5 }, (_, i) => currentDate.getFullYear() - 2 + i);

// Render reports page with filters and charts
return (
  <Box sx={{ mt: 4 }}>
    <Typography variant="h5" sx={{ mb: 3, fontWeight: 'bold' }}>Reports</Typography>
    /* Filters Section for month/year/currency selection */
    <Paper elevation={3} sx={{ p: 3, mb: 4 }}>
      <Grid container spacing={2} sx={{ mb: 2 }}>
        <Grid item xs={12} sm={4}>
          <TextField
            select
            label="Month"
            value={month}
            onChange={(e) => setMonth(Number(e.target.value))}
            fullWidth
          >
            {Array.from({ length: 12 }, (_, i) => i + 1).map((m) => (
              <MenuItem key={m} value={m}>{monthNames[m]}</MenuItem>
            ))}
          </TextField>
        </Grid>
        <Grid item xs={12} sm={4}>
          <TextField
            select
            label="Year"
            value={year}
            onChange={(e) => setYear(Number(e.target.value))}
            fullWidth
          >
            {years.map((y) => (
              <MenuItem key={y} value={y}>{y}</MenuItem>
            ))}
          </TextField>
        </Grid>
        <Grid item xs={12} sm={4}>
          <TextField
            select
            label="Currency"
            value={currency}
            onChange={(e) => setCurrency(e.target.value)}
            fullWidth
          >
            {currencies.map((c) => (
              <MenuItem key={c} value={c}>{c}</MenuItem>
            ))}
          </TextField>
        </Grid>
      </Grid>
    </Paper>
  </Box>
);

```

```

    )))
    </TextField>
  </Grid>
</Grid>
</Grid>
{error && <Typography color="error" sx={{ mb: 2 }}>{error}</Typography>}
</Paper>
/* Charts Section - displays pie and bar charts */
{reportData && yearlyData && (
  <Grid container spacing={4}>
    <Grid item xs={12} md={6}>
      <Paper elevation={2} sx={{ p: 3 }}>
        <Typography variant="h6" sx={{ mb: 2 }}>
          Costs by Category - {monthNames[month]} {year}
        </Typography>
        <PieChart data={reportData.costs} currency={currency} />
      </Paper>
    </Grid>
    <Grid item xs={12} md={6}>
      <Paper elevation={2} sx={{ p: 3 }}>
        <Typography variant="h6" sx={{ mb: 2 }}>
          Monthly Costs - {year}
        </Typography>
        <BarChart data={yearlyData} currency={currency} year={year} />
      </Paper>
    </Grid>
  </Grid>
)}
/* Summary Section showing total expenses and item count */
{reportData && (
  <Paper elevation={2} sx={{ p: 3, mt: 4 }}>
    <Typography variant="h6" sx={{ mb: 2 }}>
      Summary for {monthNames[month]} {year}
    </Typography>
    <Typography variant="body1">
      Total Expenses: <strong>{currency} {reportData.total.sum.toFixed(2)}</strong>
    </Typography>
    <Typography variant="body2" sx={{ mt: 1, color: 'text.secondary' }}>
      Items: {reportData.costs.length}
    </Typography>
  </Paper>
)}
</Box>
);
};

export default Reports;

`
## settings.js
`javascript

import React, { useState, useEffect } from 'react';
import { Paper, TextField, Button, Box, Typography, Alert } from '@mui/material';
import exchangeRateService from '../services/exchange_rate_service';

const Settings = () => {
  // State for exchange rate URL input
  const [exchangeRateURL, setExchangeRateURL] = useState('');
  // State for displaying success/error messages
  const [message, setMessage] = useState(null);
  // State for button loading during URL validation
  const [loading, setLoading] = useState(false);

  // Load saved URL on component mount
  useEffect(() => {
    const saved = exchangeRateService.getExchangeRateURL();
    if (saved) {
      setExchangeRateURL(saved);
    }
  }, []);

  // Handle saving and validating the custom exchange rate URL
  const handleSaveURL = async () => {
    // If URL is empty, clear custom settings and use defaults
    if (!exchangeRateURL.trim()) {
      exchangeRateService.setExchangeRateURL(null);
      setMessage({ type: 'success', text: 'Settings cleared. Using default exchange rates.' });
      return;
    }

    // Validate URL format using built-in URL constructor
    try {
      new URL(exchangeRateURL);
    } catch (err) {
      setMessage({ type: 'error', text: 'Invalid URL format. Please enter a valid URL.' });
      return;
    }
  };
};

```

```

    }

    // Attempt to fetch and validate the exchange rate data
    setLoading(true);
    try {
      const response = await fetch(exchangeRateURL);
      if (!response.ok) {
        throw new Error(`HTTP error! status: ${response.status}`);
      }
      const data = await response.json();

      // Validate response contains all required currency rates
      if (!data.USD || !data.GBP || !data.EURO || !data.ILS) {
        throw new Error('Response missing required currency rates (USD, GBP, EURO, ILS)');
      }

      // Save the URL if validation succeeds
      exchangeRateService.setExchangeRateURL(exchangeRateURL);
      setMessage({ type: 'success', text: 'Exchange rate URL saved and verified successfully!' });
    } catch (error) {
      // Show error details if validation fails
      setMessage({ type: 'error', text: `Failed to verify URL: ${error.message}` });
    } finally {
      setLoading(false);
    }
  }
};

// Handle clearing the URL and reverting to defaults
const handleClear = () => {
  setExchangeRateURL('');
  exchangeRateService.setExchangeRateURL(null);
  setMessage({ type: 'success', text: 'Settings cleared. Using default exchange rates.' });
};

// Render settings form UI
return (
  <Paper elevation={3} sx={{ p: 4, maxWidth: 600, mx: 'auto', mt: 5 }}>
    <Typography variant="h5" gutterBottom>Settings</Typography>
    <Typography variant="body2" color="textSecondary" sx={{ mb: 3 }}>
      Configure the URL for custom currency exchange rates. Leave empty to use default rates.
    </Typography>
    <Box sx={{ mb: 3 }}>
      <Typography variant="subtitle2" sx={{ mb: 1 }}>
        Expected Response Format:
      </Typography>
      <Typography variant="caption" sx={{ fontFamily: 'monospace', display: 'block', p: 1, backgroundColor: '#f5f5f5',
borderRadius: 1 }}>
        {`{"USD":1, "GBP":0.73, "EURO":0.92, "ILS":3.65}`}
      </Typography>
    </Box>
    {message && (
      <Alert severity={message.type} sx={{ mb: 3 }}>
        {message.text}
      </Alert>
    )}
    <Box sx={{ display: 'flex', flexDirection: 'column', gap: 2 }}>
      <TextField
        label="Exchange Rate URL"
        type="url"
        placeholder="https://example.com/exchange-rates"
        value={exchangeRateURL}
        onChange={(e) => setExchangeRateURL(e.target.value)}
        fullWidth
        multiline
        rows={2}
        helperText="Enter a complete URL that returns JSON with exchange rates"
      />
      <Box sx={{ display: 'flex', gap: 2, mt: 2 }}>
        <Button variant="contained" onClick={handleSaveURL} disabled={loading} sx={{ flex: 1 }}>
          {loading ? 'Verifying...' : 'Save & Verify'}
        </Button>
        <Button variant="outlined" onClick={handleClear} sx={{ flex: 1 }}>
          Clear
        </Button>
      </Box>
    </Box>
  </Paper>
);
};

export default Settings;

`n

## bar_chart.js
`javascript

```



```

import React, { useEffect, useRef } from 'react';
import * as d3 from 'd3';

const BarChart = ({ data, currency, year }) => {
  // Reference to SVG container for D3 rendering
  const svgRef = useRef();

  // Generate and update chart whenever data changes
  useEffect(() => {
    // Skip rendering if no data provided
    if (!data) return;

    // Month abbreviations for X-axis labels
    const monthNames = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];
    // Convert data object to array with month labels
    const chartData = Array.from({ length: 12 }, (_, i) => ({
      month: monthNames[i],
      monthNumber: i + 1,
      sum: parseFloat((data[i + 1] || 0).toFixed(2))
    }));

    // Define SVG dimensions with margins for axes and labels
    const margin = { top: 20, right: 20, bottom: 30, left: 60 };
    const width = 500 - margin.left - margin.right;
    const height = 300 - margin.top - margin.bottom;

    // Clear previous chart elements to prevent duplication
    d3.select(svgRef.current).selectAll("*").remove();

    // Create SVG container with margins
    const svg = d3.select(svgRef.current)
      .attr('width', width + margin.left + margin.right)
      .attr('height', height + margin.top + margin.bottom)
      .append('g')
      .attr('transform', `translate(${margin.left},${margin.top})`);

    // Create X-axis scale for months
    const xScale = d3.scaleBand()
      .domain(chartData.map(d => d.month))
      .range([0, width])
      .padding(0.1);

    // Create Y-axis scale with padding for better visualization
    const maxSum = d3.max(chartData, d => d.sum) || 1;
    const yScale = d3.scaleLinear()
      .domain([0, maxSum * 1.1])
      .range([height, 0]);

    // Define color gradient based on values
    const colorScale = d3.scaleLinear()
      .domain([0, maxSum])
      .range(['#E3F2FD', '#1976D2']);

    // Draw bars for each month with color based on value
    svg.selectAll('.bar')
      .data(chartData)
      .enter()
      .append('rect')
      .attr('class', 'bar')
      .attr('x', d => xScale(d.month))
      .attr('y', d => yScale(d.sum))
      .attr('width', xScale.bandwidth())
      .attr('height', d => height - yScale(d.sum))
      .attr('fill', d => colorScale(d.sum))
      .attr('stroke', '#1976D2')
      .attr('stroke-width', 1);

    // Add value labels on top of each bar
    svg.selectAll('.bar-label')
      .data(chartData)
      .enter()
      .append('text')
      .attr('class', 'bar-label')
      .attr('x', d => xScale(d.month) + xScale.bandwidth() / 2)
      .attr('y', d => yScale(d.sum) - 5)
      .attr('text-anchor', 'middle')
      .attr('font-size', '11px')
      .attr('fill', '#333')
      .text(d => d.sum > 0 ? `${currency} ${d.sum.toFixed(0)}` : '');

    // Draw X-axis with month labels
    svg.append('g')
      .attr('transform', `translate(0,${height})`)
      .call(d3.axisBottom(xScale))
      .attr('font-size', '12px');
  });
};

```

```

    // Draw Y-axis with value scale
    svg.append('g')
      .call(d3.axisLeft(yScale))
      .attr('font-size', '12px');

    // Add Y-axis label showing currency
    svg.append('text')
      .attr('transform', 'rotate(-90)')
      .attr('y', 0 - margin.left)
      .attr('x', 0 - (height / 2))
      .attr('dy', '1em')
      .attr('text-anchor', 'middle')
      .attr('font-size', '12px')
      .text(`Amount (${currency})`);

    // Add grid lines for easier value reading
    svg.append('g')
      .attr('class', 'grid')
      .attr('opacity', 0.1)
      .call(d3.axisLeft(yScale)
        .tickSize(-width)
        .tickFormat('')
      );

    }, [data, currency, year]);

    // Render SVG container (D3 will populate via useEffect)
    return <svg ref={svgRef}></svg>;
  };
}

export default BarChart;

``n
## pie_chart.js
`javascript

import React, { useEffect, useRef } from 'react';
import * as d3 from 'd3';

const PieChart = ({ data, currency }) => {
  // Reference to SVG container for D3 rendering
  const svgRef = useRef();

  // Generate and update chart whenever data or currency changes
  useEffect(() => {
    // Always clear previous chart to prevent stale data
    d3.select(svgRef.current).selectAll("*").remove();

    // Skip rendering if no data provided
    if (!data || data.length === 0) return;
    const categoryTotals = {};
    data.forEach(item => {
      if (!categoryTotals[item.category]) {
        categoryTotals[item.category] = 0;
      }
      categoryTotals[item.category] += item.convertedSum;
    });

    // Convert to array format required by D3 pie generator
    const chartData = Object.entries(categoryTotals).map(([category, sum]) => ({
      category,
      sum: parseFloat(sum.toFixed(2))
    }));

    // Exit if no categories have data
    if (chartData.length === 0) return;

    // Set SVG dimensions for responsive layout
    const width = 600;
    const height = 300;
    const radius = Math.min(width, height) / 2 - 10;

    // Create SVG container and central group for pie
    const svg = d3.select(svgRef.current)
      .attr('width', width)
      .attr('height', height)
      .append('g')
      .attr('transform', `translate(${width / 2},${height / 2})`);

    // Define color scheme for pie slices
    const color = d3.scaleOrdinal(d3.schemeCategory10);

    // Create pie layout generator from data
    const pie = d3.pie().value(d => d.sum);

```

```

    // Create arc generator for slice paths
    const arc = d3.arc().innerRadius(0).outerRadius(radius);

    // Bind data to pie slices and create groups
    const arcs = svg.selectAll('.arc')
      .data(pie(chartData))
      .enter()
      .append('g')
      .attr('class', 'arc');

    // Draw colored paths for each slice
    arcs.append('path')
      .attr('d', arc)
      .attr('fill', (d, i) => color(i))
      .attr('stroke', 'white')
      .attr('stroke-width', 2);

    // Add percentage labels on pie slices
    arcs.append('text')
      .attr('transform', d => `translate(${arc.centroid(d)})`)
      .attr('text-anchor', 'middle')
      .attr('font-size', '12px')
      .attr('font-weight', 'bold')
      .attr('fill', 'white')
      .text(d => {
        // Calculate percentage of total for display
        const percentage = ((d.data.sum / d3.sum(chartData, d => d.sum)) * 100).toFixed(1);
        return `${percentage}%`;
      });

    // Create legend showing category names and amounts
    const legend = svg.selectAll('.legend')
      .data(chartData)
      .enter()
      .append('g')
      .attr('class', 'legend')
      .attr('transform', (d, i) => `translate(${radius + 40},${-radius + i * 25})`);

    // Draw larger colored boxes for legend entries
    legend.append('rect')
      .attr('width', 16)
      .attr('height', 16)
      .attr('fill', (d, i) => color(i))
      .attr('stroke', '#333')
      .attr('stroke-width', 1);

    // Add category name clearly next to color box
    legend.append('text')
      .attr('x', 22)
      .attr('y', 12)
      .attr('font-size', '13px')
      .attr('font-weight', 'bold')
      .attr('fill', '#333')
      .text(d => d.category);

    // Add amount below category name
    legend.append('text')
      .attr('x', 22)
      .attr('y', 26)
      .attr('font-size', '11px')
      .attr('fill', '#666')
      .text(d => `${currency} ${d.sum.toFixed(2)}`);

    }, [data, currency]);

    // Render SVG container (D3 will populate via useEffect)
    return <svg ref={svgRef}></svg>;
  };
}

export default PieChart;

``
## exchange_rate_service.js
`javascript

// services/exchangeRateService.js - Handles currency exchange rate fetching and conversion

// Default exchange rates (USD as base)
const DEFAULT_RATES = {
  'USD': 1,
  'GBP': 0.73,
  'EURO': 0.92,
  'ILS': 3.65
};

// Get stored custom exchange rate URL from localStorage

```

```

const getExchangeRateURL = () => {
  return localStorage.getItem('exchangeRateURL');
};

// Set custom exchange rate URL in localStorage
const setExchangeRateURL = (url) => {
  if (url) {
    localStorage.setItem('exchangeRateURL', url);
  } else {
    localStorage.removeItem('exchangeRateURL');
  }
};

// Fetch exchange rates from custom URL or use defaults with fallback strategy
const fetchExchangeRates = async () => {
  try {
    // Retrieve custom URL if user has configured one
    const customURL = getExchangeRateURL();
    let url = customURL;

    // If no custom URL, try default backend server first
    if (!url) {
      url = process.env.REACT_APP_EXCHANGE_RATE_API || 'https://cost-manager-0w4h.onrender.com/api/exchange-rates';
    }

    // Attempt to fetch from configured URL (custom or default)
    try {
      const response = await fetch(url);
      if (response.ok) {
        const data = await response.json();
        // Validate that response contains required currencies
        if (data.USD && data.GBP && data.EURO && data.ILS) {
          // Cache rates in localStorage for offline use
          localStorage.setItem('cachedExchangeRates', JSON.stringify(data));
          return data;
        }
      }
    } catch (fetchError) {
      console.warn('Error fetching from server:', fetchError);
    }

    // Check if we have previously cached rates as fallback
    const cached = localStorage.getItem('cachedExchangeRates');
    if (cached) {
      return JSON.parse(cached);
    }

    // Final fallback: use hardcoded default rates
    localStorage.setItem('cachedExchangeRates', JSON.stringify(DEFAULT_RATES));
    return DEFAULT_RATES;
  } catch (error) {
    // Return cached rates or defaults
    console.warn('Error fetching exchange rates, using cached or default:', error);
    const cached = localStorage.getItem('cachedExchangeRates');
    return cached ? JSON.parse(cached) : DEFAULT_RATES;
  }
};

// Convert amount from one currency to another using fetched rates
const convertCurrency = (amount, fromCurrency, toCurrency, rates) => {
  // No conversion needed if currencies are the same
  if (fromCurrency === toCurrency) return amount;

  // Validate currencies exist in rates table
  if (!rates[fromCurrency] || !rates[toCurrency]) {
    console.error('Invalid currency:', fromCurrency, toCurrency);
    return amount;
  }

  // Convert to USD first (base currency), then to target currency
  const amountInUSD = amount / rates[fromCurrency];
  return amountInUSD * rates[toCurrency];
};

// Export service functions for use in components
export default {
  getExchangeRateURL,
  setExchangeRateURL,
  fetchExchangeRates,
  convertCurrency,
  DEFAULT_RATES
};

```